



UNIT - II

REAL-TIME

DATA

PROCESSING



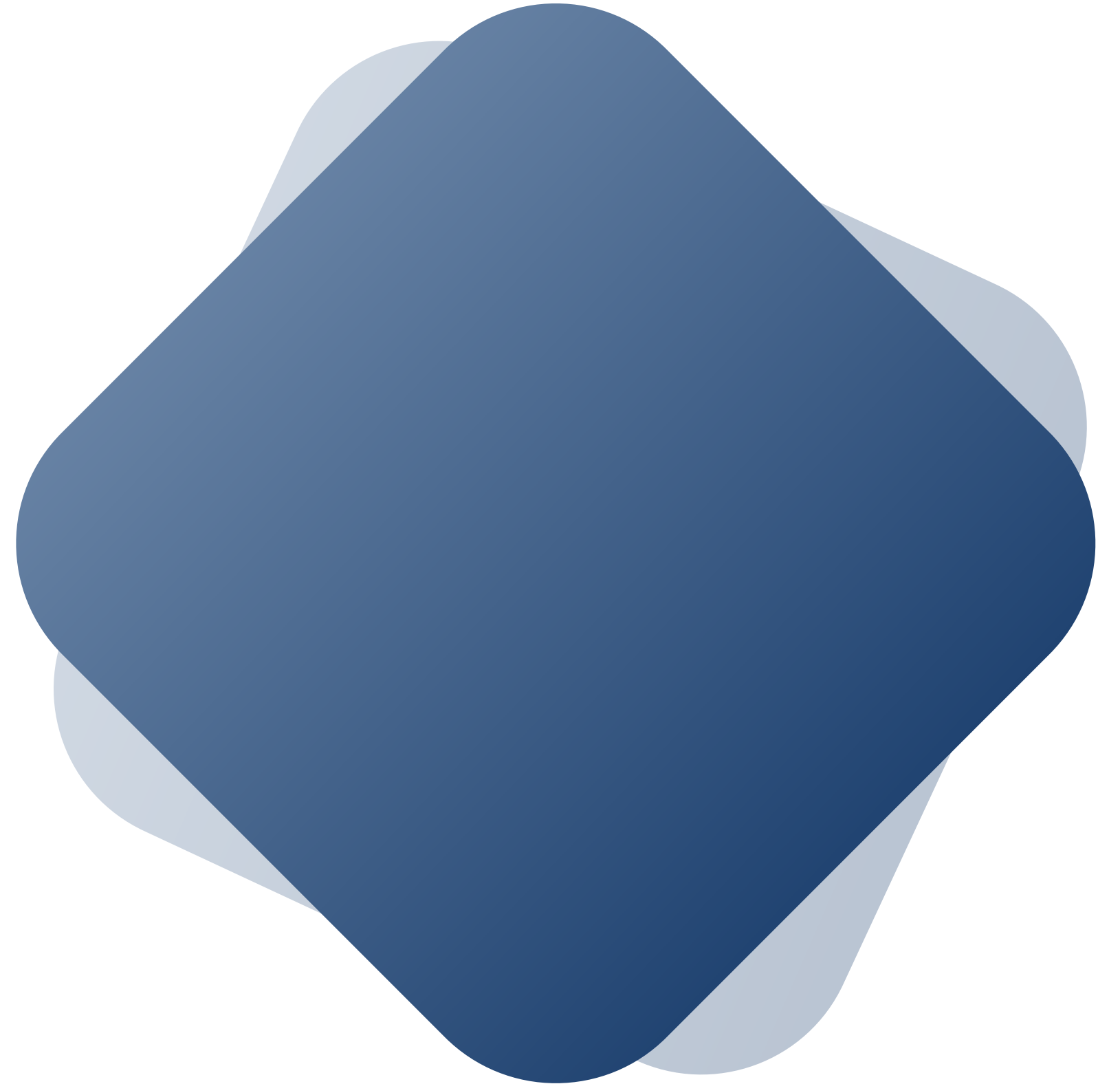
UNIT II REAL-TIME DATA PROCESSING

6

Introduction to Big data, Big data infrastructure, Real-time Analytics, Near real-time solution, Lambda architecture, Kappa Architecture, Stream Processing, Understanding Data Streams, Message Broker, Stream Processor, Batch & Real-time ETL tools, Streaming Data Storage

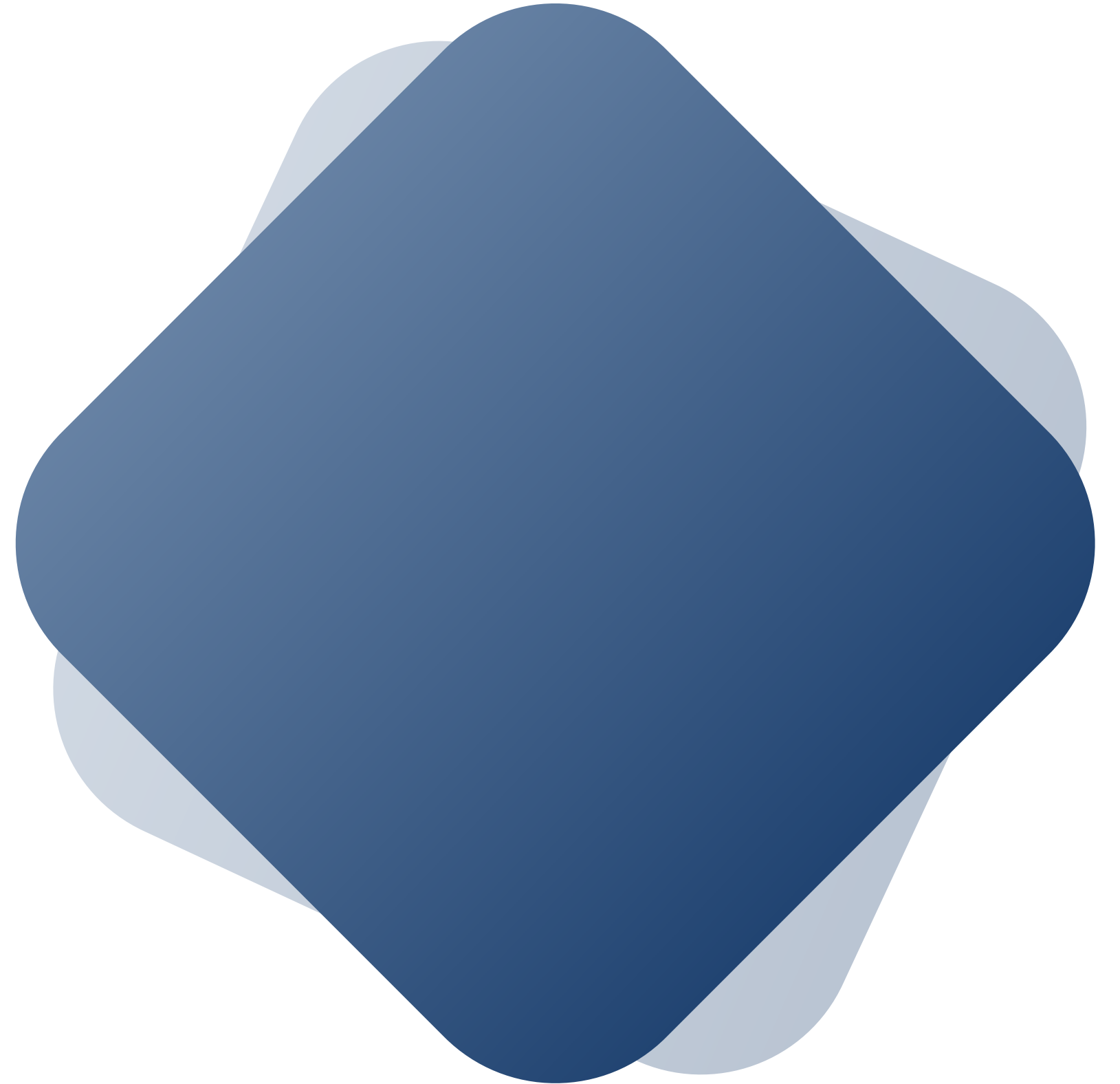
Real-Time Analytics

- Real-time analytics is a set of techniques for processing data as soon as it becomes available.
- The main goal is to provide fast and actionable insights.



Real-Time Analytics

- Companies must derive real-time insights from a variety of sources that produce large volumes of data at an extremely high rate.
- One of the biggest challenges of real-time analytics is **combining data from different sources and analyzing it in a timely manner.**



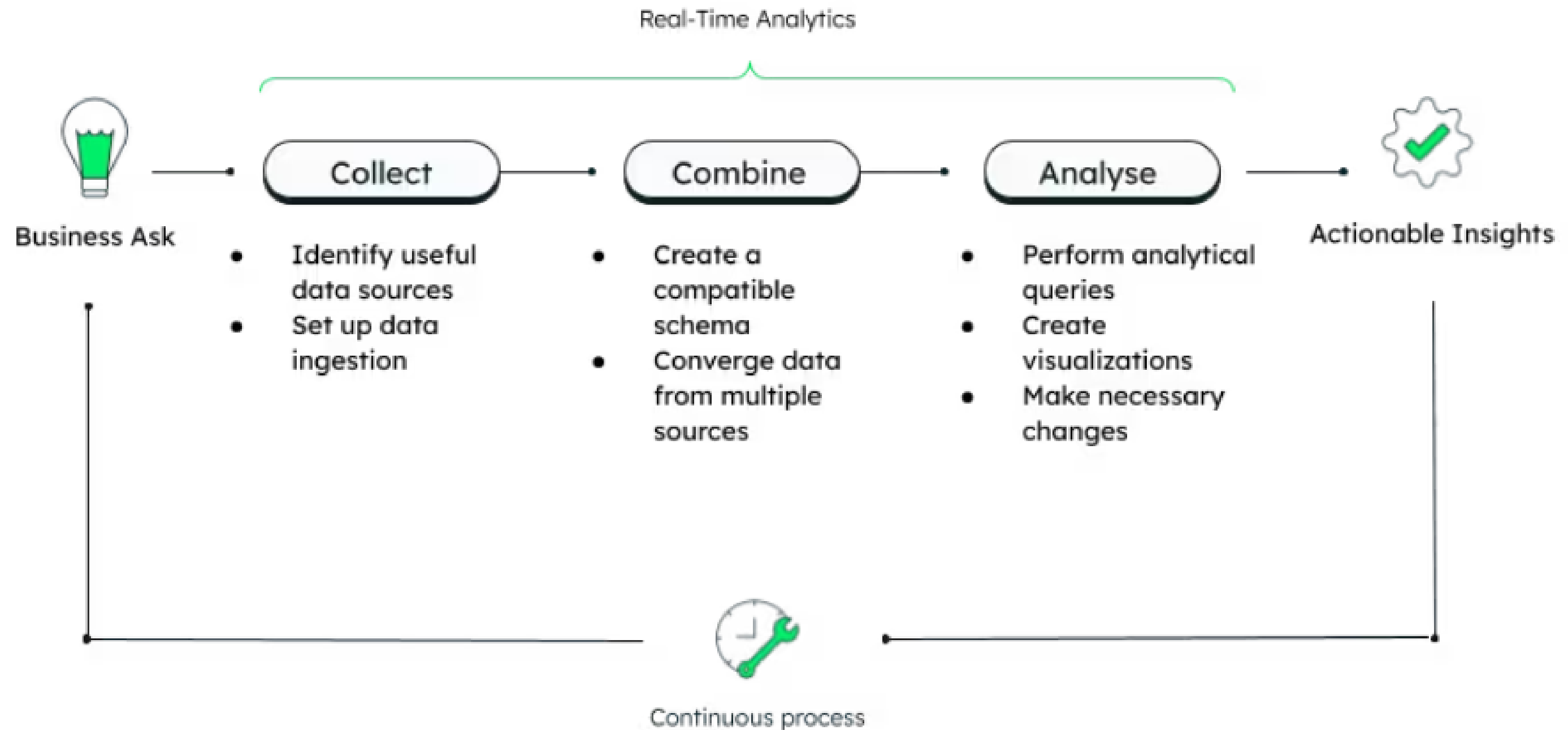
Real-time Analytics Examples

- Providing the customer with an offer or a piece of information that matches their needs and inclinations based on a real-time analysis of their behavior.
- Application monitoring to prevent downtime and improve performance.
- Real-time blocking of fraudulent transactions.
- Chatbots that automate conversations with customers.

How Does Real-Time Analytics Work?

- Real-time: Data is collected and processed rapidly and continuously.
- Analytics: Actionable insights are extracted by applying a set of algorithms to the data.

How Does Real-Time Analytics Work?



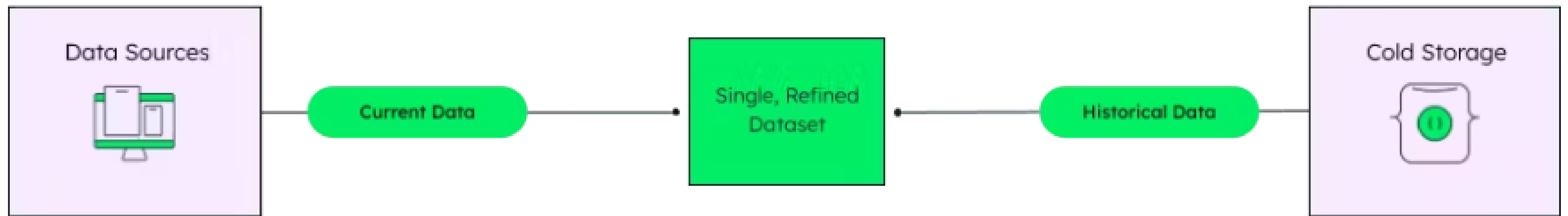
Real-time analytics process: Collect data, combine sources, analyze to extract actionable insights

Collect useful data

- Real-time analytics only works if you're collecting useful data as it becomes available.
- You need to understand what data is important for your business and how it can be collected.
- This is the first step in the process.
- For example, you may be running a manufacturing company. You need to know if a machine is showing signs of failure or if it's working perfectly.
- To do that, you need to collect data from the **machine sensors** and monitor it in real time.
- Integrated data platforms that provide real-time data ingestion allow you to collect useful data as it becomes available.
- **Apache Kafka** is a well-established solution for implementing real-time data transfer with event streams.

Combine data from multiple sources

- In practice, we usually collect data from multiple sources.
- In our machine-monitoring example, we may have a sensor that collects data about the **temperature of the machine**, a sensor that collects data about the **pressure of the machine**, and a sensor that collects data about the **humidity of the machine**.
- To perform a complete analysis, we need to combine the data from all of these sources.
- The second part of the real-time analytics process is the convergence of data from multiple sources.



Combining current and historical data leads to more complete real time analysis

Analyze to Extract Insights

- The third and final step in the process is to extract actionable insights from the data.
- This is where the real-time analytics process really starts to make sense.
- But to analyze data, you need to have the right tools.
- Different tools solve that problem differently.

Real-time Analytics Use Cases

- **Personalization & Experience** - designing products or producing services and products to meet customers' individual requirements.
- **Fraud & Error Prevention**
- **Process Optimization**
- **Preemptive Maintenance** - Pre-emptive maintenance is a necessary process to ensure that your assets are running smoothly.
- Without pre-emptive maintenance, you can run into many issues, including asset downtime and missed deadlines.
- Therefore, you need a way to track important asset dates and set reminders for maintenance.

Real-time, near real-time, and batch processing

Type of data processing

When do you need it?

Real-time

When you need information processed immediately (such as at a bank ATM)

Near real-time

When speed is important, but you don't need it immediately (such as producing operational intelligence)

Batch

When you can wait for days (or longer) for processing (Payroll is a good example.)

Examples of Near Real- Time Processing

- **Processing sensor data**
- **IT systems monitoring**
- **Financial transaction processing**

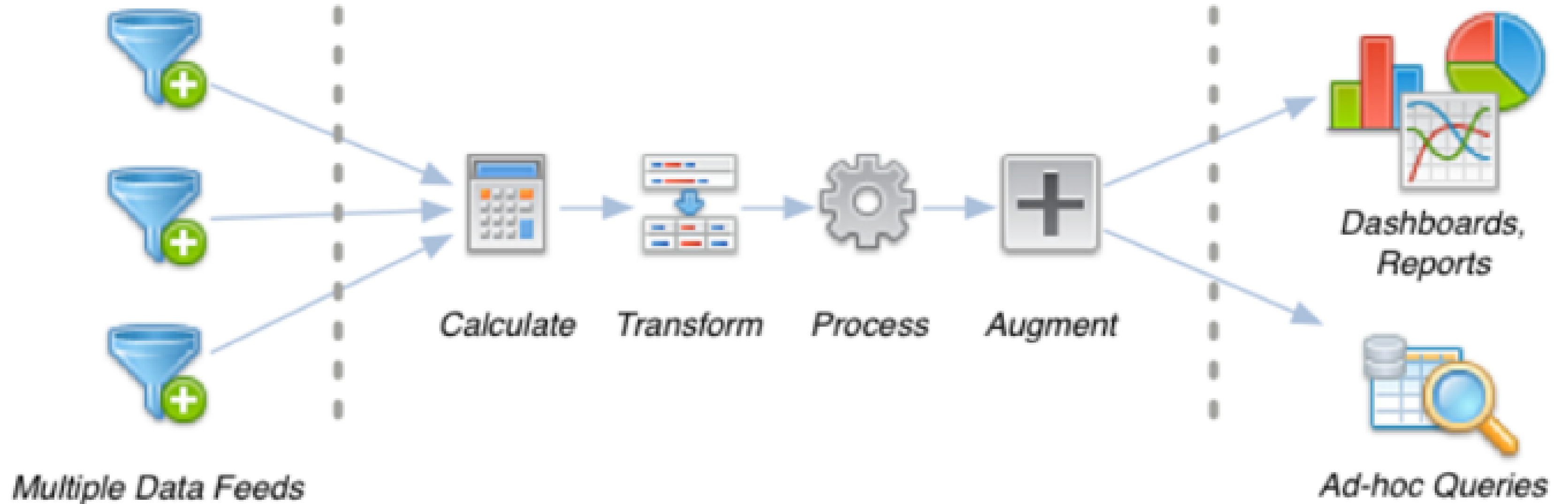
Near Real-time Solution

- A high–level NRT solution recipe looks very straight and simple, with a data collection funnel, a distributed processing engine, and a few other ingredients like in–memory cache, stable storage, and dashboard plugins.
- **Real–time data collection** of the streaming data
- **Distributed** **high–performance computation** on flowing data
- **Exploring and visualizing** the generated insights in the form of query–able consumable layer/dashboards

(1) Collect real-time data.

(2) Process data as it flows.

(3) Explore and visualize.



Near Real-time Solution

- One of the very recent NRT architectural patterns is **Lambda architecture**, which is a most sought after solution that combines the best of both **batch and real-time** implementations, without having any need to replicate and maintain two solutions.

STREAM PROCESSING ARCHITECTURES

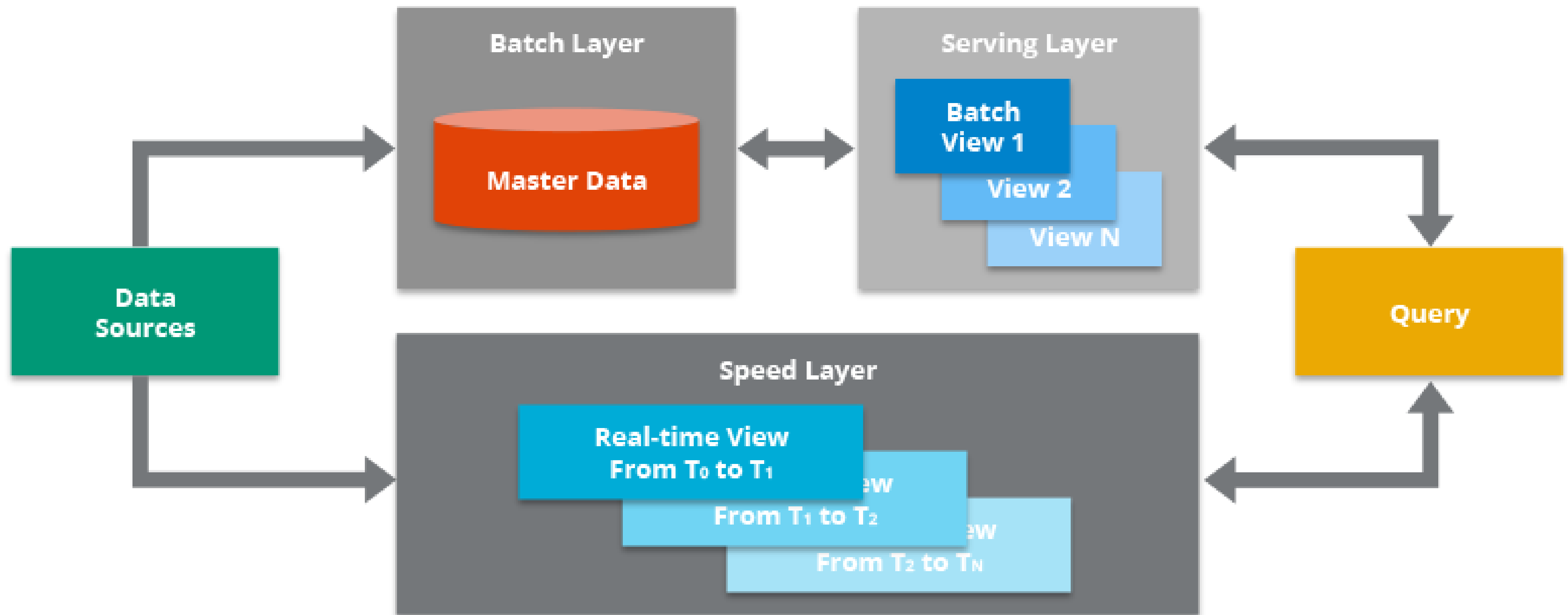
1. LAMBDA ARCHITECTURE

2. KAPPA ARCHITECTURE

Lambda Architecture

- Lambda architecture is a data processing approach designed to handle large volumes of data efficiently.
- It combines two processing methods:
 1. **Batch processing:** This layer handles historical data in its entirety, offering comprehensive and accurate views. It processes the data periodically, like overnight, ensuring high data quality but potentially experiencing higher latency.
 2. **Real-time processing (speed layer):** This layer focuses on recent data streams, providing near real-time insights with lower latency. While faster, this layer might have slightly less accurate data compared to the batch layer.

LAMBDA ARCHITECTURE



Lambda Architecture

- Lambda architecture consists of three main layers:

1. Batch layer:

- This layer processes all the data using batch jobs, typically scheduled to run periodically (e.g., daily).
- It utilizes tools like **ETL (Extract, Transform, Load)** processes to clean, transform, and load data into a data warehouse or other storage system.
- This layer ensures data consistency and accuracy for historical analysis.

Lambda Architecture

2. Speed layer (real-time processing):

- This layer processes data streams in real time with lower latency.
- It utilizes technologies like **Apache Kafka** or **Apache Spark Streaming** to ingest and process data as it arrives.
- This layer provides near real-time insights but may have slightly less data accuracy compared to the batch layer.

Lambda Architecture

3. Serving layer:

- This layer acts as the **unified interface for querying and accessing data**.
- It combines the results from both the batch and speed layers, potentially after reconciling any discrepancies.
- The serving layer uses technologies like **search engines or data visualization tools** to enable users to analyze the data effectively.

Lambda Architecture

Immutable Master Copy:

- The Lambda architecture typically utilizes an immutable master copy of the entire data set.
- This means the data is never modified, and new data is appended to the existing data.
- This approach simplifies data management and ensures data consistency across layers.
- Overall, the architecture offers a flexible and scalable approach for handling diverse data processing needs, from historical analysis to real-time insights.

Benefits of Lambda Architecture

- **Handles diverse data needs:** It caters to both historical data analysis (batch layer) and real-time needs (speed layer).
- **Scalability:** The architecture scales well for handling massive data volumes.
- **Fault tolerance:** Both layers are designed to be fault-tolerant, ensuring data processing continuity even during failures.

Kappa Architecture

- Kappa architecture is an alternative to Lambda architecture for processing streaming data.
- The Kappa architecture is a data processing approach designed for **real-time and batch processing using a single technology stack**.
- It simplifies the architecture by focusing on a single processing path based on stream processing technologies.
- Unlike Lambda's distinct batch and real-time layers, Kappa **processes** all data as a **continuous stream**.

when to choose Kappa Architecture?

- Kappa shines in situations where:
 - Real-time analytics is a critical component.
 - Simplified data architecture is desired.
 - Both real-time and batch processing are needed with consistent results.
 - Historical data reprocessing might be necessary in the future.

Kappa Philosophy: Streaming all the things

- Kappa architecture centers around the idea that all data can be treated as a continuous, unbounded stream. **Key principles** include:
- **Stream-First Mentality:** Incoming data, whether new or historical, is ingested as a stream.
- **Immutable Data:** Raw data is stored as-is and never modified, ensuring a reliable source of truth.
- **Code as Data:** Computations and transformations are applied to streams, and the code itself becomes part of the stored data lineage, enabling reprocessing if logic changes.

Components of Kappa Architecture

1. Data Ingestion
2. Stream Processing Engine
3. Messaging Engine
4. Serving Layer (Data Store)
5. Analytics Applications

Data Ingestion

Data enters the system from various sources, like sensors, applications, or databases.

Data Sources: These can be anything that generates data:

- **Event logs from web servers**
- **IoT device sensors**
- **Database change logs**
- **Transactional systems**

Messaging Engine

- It is based on a streaming architecture in which an incoming series of data is first stored in a messaging engine like Apache Kafka.
- Message engine acts as a **buffer** between the data source and the SPE.
- It ensures **scalability** and **Fault Tolerance**.

Stream Processing Engine

- The **core/heart** of Kappa, the SPE ingests the continuous stream of data from the source.
- It reads the incoming data stream, performs data cleansing, transformations, and aggregations, and transforms the data into a **format suitable for analysis**.
- Examples of SPEs include Apache Flink, Apache Spark Streaming, and Kafka Streams.

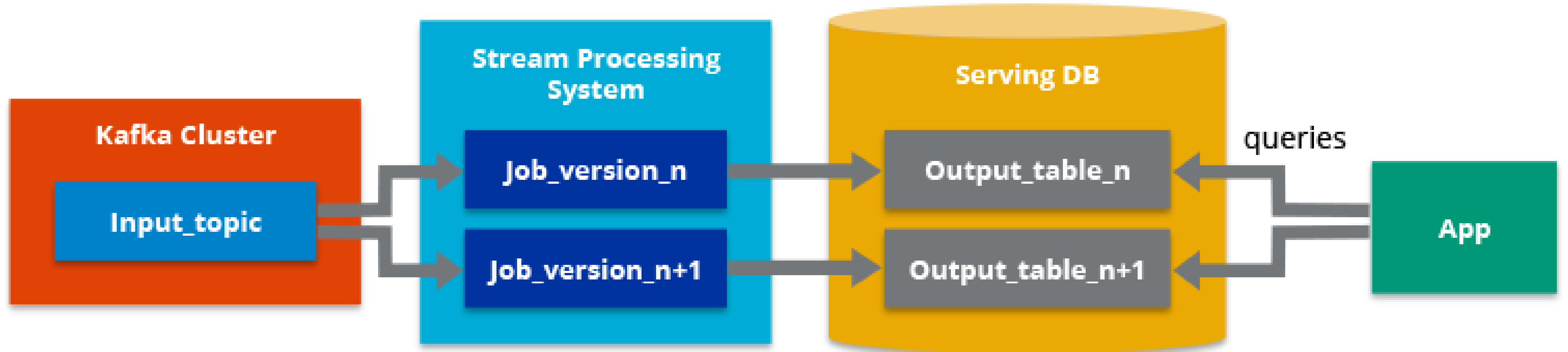
Serving Layer (Data Store)

- The processed data is stored in a distributed and scalable data store optimized for both real-time queries and analysis of historical data. Common choices include:
 - **NoSQL databases (HBase, Cassandra) for real-time access**
 - **Distributed File Systems (HDFS) for cost-effective storage of large volumes**
 - **Cloud Object Stores (Amazon S3) for durability and cost efficiency**

Analytics Applications

- Applications access the data in the store for real-time analytics (using recent data) or batch analytics (using historical data).
- Data analysts, dashboards, reporting tools, and machine learning models interact with the serving layer to derive insights and actions based on the processed data.

KAPPA ARCHITECTURE



- Data enters a central data queue such as Apache Kafka and is converted into a format that can be directly fed into an analytics database.
- By removing complexity and increasing efficiency, this unified method enables you to analyze data more quickly and instantly obtain deeper insights.

Key Characteristics of Kappa Architecture

- **Real-time focus:** Processes all data in real-time, enabling immediate insights and quicker reactions to events.
- **Simplified architecture:** Uses a **single processing pipeline** with a stream processing engine like Apache Flink or Apache Spark Streaming, reducing complexity compared to **Lambda's dual layers**.
- **Unified data store:** Stores all data in a single store, often a distributed file system like Apache HDFS or a streaming database like Apache Kafka Streams. This allows for historical and real-time analysis from the same source.

Benefits of Kappa Architecture

- **Simplicity:** Single processing system reduces complexity compared to Lambda.
- **Real-time Insights:** Enables near real-time analytics on recent data.
- **Flexibility:** Supports both real-time and batch processing needs.
- **Scalability:** Handles large data volumes efficiently due to distributed components.

Challenges of Kappa Architecture

- **Error Handling:** Requires careful design to handle errors and ensure data consistency.
- **Debugging:** Debugging issues can be more complex due to the single processing pipeline.
- **Latency:** Real-time processing might introduce slight latency compared to dedicated real-time systems.

Kappa Architecture vs Lambda Architecture

- The Kappa Architecture is considered a simpler alternative to the Lambda Architecture as it uses the **same technology stack** to handle both real-time stream processing and historical batch processing.
- Both architectures entail the **storage of historical data** to **enable large-scale analytics**.
- Both architectures are also useful for addressing “**human fault tolerance,**” in which problems with the processing code (either **bugs** or just known limitations) can be overcome by **updating the code and running it again** on the historical data.
- The main difference with the Kappa Architecture is that all data is treated as if it were a stream, so the stream processing engine acts as the sole data transformation engine.